

**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

Laboratory Work No. 5

Operating Systems

The File System with Your Own Hands

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Contents

1 Objective **2**

2 Laboratory Environment **2**

3 Part 1 – The Tree and File Types **3**
3.1 Exploring the Linux Directory Tree . . . 3
3.2 Identifying File Types . . . 3
3.3 Character and Block Devices . . . 4
3.4 Identifying File Contents . . . 4
3.5 Comparing Directory Sizes . . . 4
3.6 Part 1 – Observations . . . 5

4 Part 2 – Inodes and Links **5**
4.1 Examining an Inode . . . 5
4.2 Creating a Hard Link . . . 6
4.3 Deleting the Original Filename . . . 7
4.4 Creating a Symbolic Link . . . 7
4.5 Broken Symbolic Link . . . 7
4.6 Directory Link Counts . . . 8
4.7 File Size and Disk Usage . . . 8
4.8 Part 2 – Observations . . . 9

5 Part 3 – Permissions **10**
5.1 Changing File Permissions . . . 10
5.2 Removing Permissions . . . 10
5.3 Examining the Umask . . . 11
5.4 Permission Modes . . . 12
5.5 Directory Read and Execute Permissions . . . 12
5.6 Special Permission Bits . . . 13
5.7 Accessing a Protected System File . . . 14
5.8 Part 3 – Observations . . . 14

6 Part 4 – Disks and the Kernel as Files **14**
6.1 Examining Disk Partitions . . . 14
6.2 Checking Disk Usage . . . 15
6.3 Identifying the Filesystem Type . . . 16
6.4 Examining CPU Information . . . 16
6.5 Examining Memory Information . . . 17
6.6 Examining Process Information . . . 17
6.7 Examining File Descriptors . . . 17
6.8 Part 4 – Observations . . . 18

7 GitHub Repository **18**

8 Conclusion **19**

1 Objective

The objective of this laboratory work was to explore the Linux file system and understand how files, directories, inodes, links, permissions and disk storage are managed by the operating system.

The laboratory was completed using Ubuntu Linux running in a VirtualBox virtual machine.

The following topics were investigated:

- Linux directory hierarchy and file types;
- character and block devices;
- inodes, hard links and symbolic links;
- directory link counts and sparse files;
- file and directory permissions;
- default permissions and the umask;
- special permission bits;
- disk partitions and mounted file systems;
- CPU, memory and process information.

All experiments were performed using a normal Linux user account without root privileges.

2 Laboratory Environment

The laboratory was performed using Ubuntu Linux.

The working directory was created with the commands:

```
whoami
mkdir -p ~/os-lab5
cd ~/os-lab5
pwd
```

The username was:

```
bianca
```

The working directory was:

```
/home/bianca/os-lab5
```

Using a normal user account was important because the permission experiments required Linux to enforce ordinary access restrictions.

3 Part 1 – The Tree and File Types

3.1 Exploring the Linux Directory Tree

The first task was to examine the top-level directories of the Linux file system.

The following command was executed:

```
ls /
```

The output included:

```
bin boot dev etc home lib
media mnt opt proc root run
sbin snap srv sys tmp usr
var
```

Linux organizes files in a hierarchical structure that begins at the root directory, represented by `/`.

Some important directories are:

- `/etc` – system configuration files;
- `/home` – user home directories;
- `/dev` – device files;
- `/usr` – applications and shared resources;
- `/tmp` – temporary files;
- `/proc` – virtual process and kernel information.

3.2 Identifying File Types

The following command was used:

```
ls -ld /etc /etc/passwd /bin /dev/null /tmp
```

The results were:

```
/bin -> usr/bin
/dev/null
/etc
/etc/passwd
/tmp
```

The first character in the permission string identifies the file type.

Path	Type	Description
/bin	l	Symbolic link
/dev/null	c	Character device
/etc	d	Directory
/etc/passwd	-	Regular file
/tmp	d	Directory

The `/bin` path is a symbolic link pointing to `/usr/bin`.

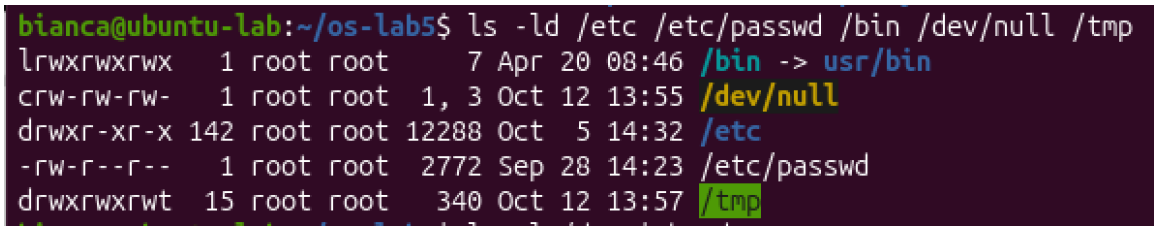


Figure 1: Identifying Linux file types and permissions

3.3 Character and Block Devices

The command:

```
ls -l /dev | head
```

displayed several device entries.
For example, `/dev/console` is a character device.
Block devices were examined using:

```
ls -l /dev/sda /dev/sda1
```

The output was:

```
brw-rw---- 1 root disk 8, 0 /dev/sda
brw-rw---- 1 root disk 8, 1 /dev/sda1
```

Character devices transfer data as streams, while block devices provide block-oriented access to storage.

3.4 Identifying File Contents

The following command was executed:

```
file /usr/bin/ls /etc/passwd /bin
```

The output was:

```
/usr/bin/ls: symbolic link to
../lib/cargo/bin/coreutils/ls

/etc/passwd: ASCII text

/bin: symbolic link to usr/bin
```

This demonstrated that the `file` command identifies the type of a file without relying only on its extension.

3.5 Comparing Directory Sizes

Disk usage was measured using:

```
du -sh /var/log 2>/dev/null
du -sh /usr 2>/dev/null
```

The screenshot showed:

```
135M /var/log
5.2G /usr
```

The `/usr` directory was significantly larger than `/var/log`.

This is because `/usr` contains installed programs, libraries and other resources, while `/var/log` mainly contains log files.

The difference was approximately 5 GB.

```
bianca@ubuntu-lab:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r-- 1 root   root     10, 235 Oct 12 13:55 autofs
drwxr-xr-x 2 root   root        420 Oct 12 13:56 block
drwxr-xr-x 2 root   root         80 Oct 12 13:55 bsg
crw------- 1 root   root     10, 234 Oct 12 13:55 btrfs-control
drwxr-xr-x 3 root   root         60 Jan  1  1970 bus
lrwxrwxrwx+1 root   root          3 Oct 12 13:55 cdrom -> sr0
drwxr-xr-x 2 root   root       3820 Oct 12 13:56 char
crw------- 1 root   tty       5,   1 Oct 12 13:55 console
lrwxrwxrwx 1 root   root         11 Oct 12 13:55 core -> /proc/kcore
bianca@ubuntu-lab:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
bianca@ubuntu-lab:~/os-lab5$ du -sh /var/log 2>/dev/null
135M    /var/log
bianca@ubuntu-lab:~/os-lab5$ du -sh /usr 2>/dev/null
5.2G    /usr
```

Figure 2: Device inspection, file identification and disk usage

3.6 Part 1 – Observations

1. The first character in the output of `ls -l` identifies whether an entry is a directory, regular file, symbolic link or device.
2. On this Ubuntu system, `/bin` is a symbolic link to `/usr/bin`.
3. An example of a character device is `/dev/null`. An example of a block device is `/dev/sda`.
4. The `/usr` directory was approximately 5 GB larger than `/var/log`.

4 Part 2 – Inodes and Links

4.1 Examining an Inode

The first task was to create a text file:

```
echo hello > notes.txt
```

The file was examined with:

```
stat notes.txt
ls -li notes.txt
```

The important results were:

```
Size: 6
Blocks: 8
IO Block: 4096
Inode: 527788
Links: 1
Access: (0664/-rw-rw-r--)
```

An inode stores information about a file, including permissions, ownership, timestamps, size and locations of its data blocks.

The inode does not store the ordinary filename as part of its metadata.

In this experiment, the inode number was **527788**.

4.2 Creating a Hard Link

A hard link was created using:

```
ln notes.txt hard.txt
ls -li notes.txt hard.txt
```

The output was:

```
527788 -rw-rw-r-- 2 ... hard.txt
527788 -rw-rw-r-- 2 ... notes.txt
```

Both filenames shared inode number **527788**.

The link count increased from 1 to 2.

This means the two directory entries referenced the same inode and file data.

```
bianca@ubuntu-lab:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 527788      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  bianca)   Gid: ( 1000/  bianca)
Access: 2026-10-12 14:06:40.347000310 +0000
Modify: 2026-10-12 14:06:40.140000310 +0000
Change: 2026-10-12 14:06:40.140000310 +0000
 Birth: 2026-10-12 14:06:40.139000310 +0000
bianca@ubuntu-lab:~/os-lab5$ ls -li notes.txt
527788 -rw-rw-r-- 1 bianca bianca 6 Oct 12 14:06 notes.txt
bianca@ubuntu-lab:~/os-lab5$ ln notes.txt hard.txt
bianca@ubuntu-lab:~/os-lab5$ ls -li notes.txt hard.txt
527788 -rw-rw-r-- 2 bianca bianca 6 Oct 12 14:06 hard.txt
527788 -rw-rw-r-- 2 bianca bianca 6 Oct 12 14:06 notes.txt
```

Figure 3: Inspecting inodes and creating a hard link

4.3 Deleting the Original Filename

The original filename was removed:

```
rm notes.txt
cat hard.txt
ls -li hard.txt
```

The result was:

```
hello
527788 -rw-rw-r-- 1 ... hard.txt
```

The file remained accessible through `hard.txt`.
This demonstrates that the `rm` command removes a directory entry.
The file data remains available while another hard link still references the inode.

4.4 Creating a Symbolic Link

A symbolic link was created:

```
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
```

The result showed:

```
527788 ... hard.txt
528075 ... soft.txt -> hard.txt
hello
```

Unlike a hard link, a symbolic link has its own inode.
In this experiment:

- `hard.txt` had inode 527788;
- `soft.txt` had inode 528075.

The symbolic link stored the path `hard.txt`.

4.5 Broken Symbolic Link

The target was deleted:

```
rm hard.txt
cat soft.txt
ls -l soft.txt
```

The output included:

```
cat: soft.txt: No such file or directory
```

However, the symbolic link still existed:

```
soft.txt -> hard.txt
```

The symbolic link became broken because its target no longer existed.
Its size was **8 bytes**, corresponding to the eight characters in the stored path `hard.txt`.

```
bianca@ubuntu-lab:~/os-lab5$ cat hard.txt
hello
bianca@ubuntu-lab:~/os-lab5$ ls -li hard.txt
527788 -rw-rw-r-- 1 bianca bianca 6 Oct 12 14:06 hard.txt
bianca@ubuntu-lab:~/os-lab5$ ln -s hard.txt soft.txt
bianca@ubuntu-lab:~/os-lab5$ ls -li hard.txt soft.txt
527788 -rw-rw-r-- 1 bianca bianca 6 Oct 12 14:06 hard.txt
528075 lrwxrwxrwx 1 bianca bianca 8 Oct 12 14:08 soft.txt -> hard.txt
bianca@ubuntu-lab:~/os-lab5$ cat soft.txt
hello
bianca@ubuntu-lab:~/os-lab5$ rm hard.txt
bianca@ubuntu-lab:~/os-lab5$ cat soft.txt
cat: soft.txt: No such file or directory
bianca@ubuntu-lab:~/os-lab5$ ls -l soft.txt
lrwxrwxrwx 1 bianca bianca 8 Oct 12 14:08 soft.txt -> hard.txt
```

Figure 4: Hard links, symbolic links and a broken link

4.6 Directory Link Counts

A directory was created:

```
mkdir d
ls -ld d
```

The initial link count was:

```
drwxrwxr-x 2 ... d
```

Two subdirectories were then created:

```
mkdir d/x d/y
ls -ld d
```

The output became:

```
drwxrwxr-x 4 ... d
```

A new directory normally has two hard links: one from its parent directory and one through its own `.` entry.
Each new subdirectory contributes another link through its `..` entry.
Therefore, the link count increased from 2 to 4.

4.7 File Size and Disk Usage

A small file was created:

```
echo x > one.txt
ls -l one.txt
du -h one.txt
```

The results were:

```
2 one.txt
4.0K one.txt
```

The logical size of the file was 2 bytes, but the allocated disk space was 4 KB. This occurs because the file system allocates disk space using blocks.

Next, a large sparse file was created:

```
truncate -s 1G big.img
ls -lh big.img
du -h big.img
```

The results were:

```
1.0G big.img
0 big.img
```

Although the logical size was 1 GB, the file occupied no allocated data blocks because its contents had not been written.

Finally, the file was removed:

```
rm big.img
```

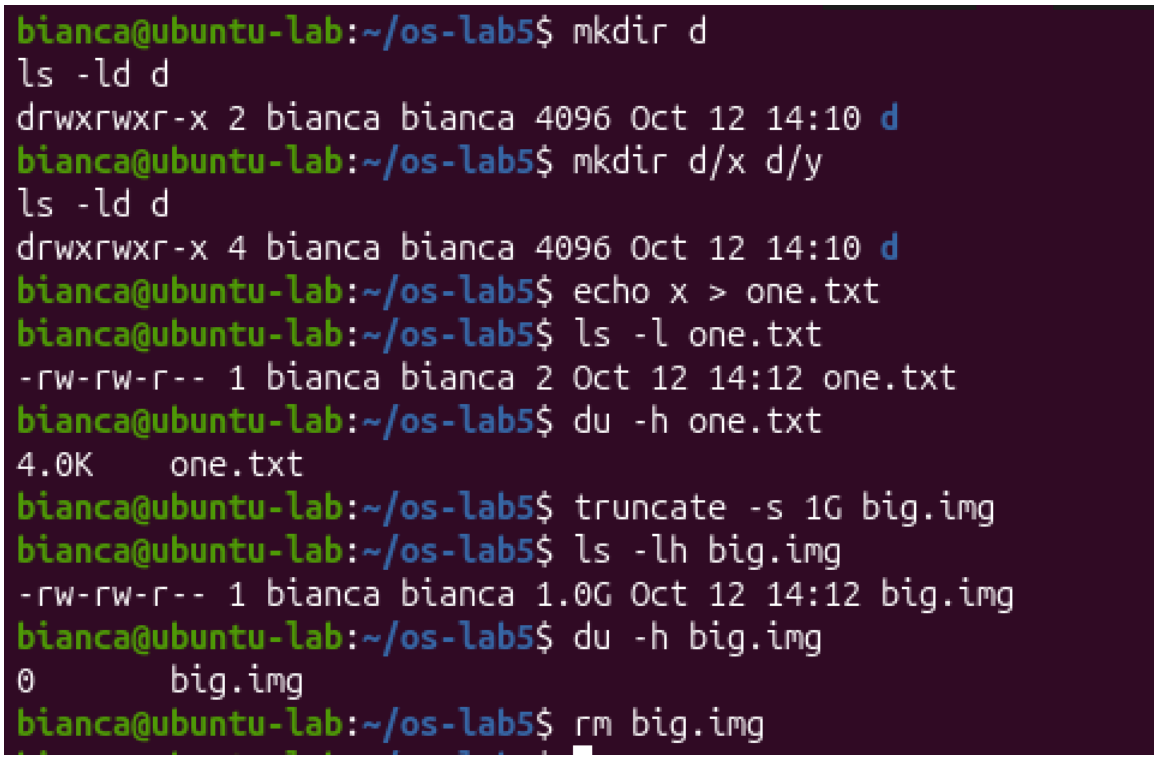


Figure 5: Directory link counts and sparse file allocation

4.8 Part 2 – Observations

1. The inode number was 527788. The `Links` field showed the number of hard links to the inode.
2. Removing `notes.txt` did not delete the data because `hard.txt` still referenced the same inode.

3. The symbolic link stopped working after its target was deleted. Its size was 8 bytes because it stored the path `hard.txt`.

4. A new directory had two links. Creating two subdirectories increased the count to four because each subdirectory referenced its parent.

5. A small file used 4 KB because of block allocation. A sparse 1 GB file occupied no data blocks because its contents were not physically written.

5 Part 3 – Permissions

5.1 Changing File Permissions

A file containing confidential text was created:

```
echo secret > s.txt
```

The permissions were changed using:

```
chmod 600 s.txt
ls -l s.txt
```

The result was:

```
-rw------- ... s.txt
```

Permission mode 600 allows the owner to read and write the file, while the group and other users have no access.

5.2 Removing Permissions

All permissions were removed:

```
chmod 000 s.txt
cat s.txt
```

The output was:

```
cat: s.txt: Permission denied
```

Although the user owned the file, the operating system denied access because the read permission was removed.

Read permission was restored:

```
chmod u+r s.txt
cat s.txt
```

The output was:

```
secret
```

This demonstrates that file ownership does not automatically bypass permission checks.

5.3 Examining the Umask

The following command was executed:

```
umask
```

The result was:

```
0002
```

A new file and directory were created:

```
touch new.txt
mkdir newdir
ls -ld new.txt newdir
```

The results were:

```
-rw-rw-r-- ... new.txt
drwxrwxr-x ... newdir
```

The permissions corresponded to:

- new file: 664;
- new directory: 775.

The umask value 0002 removes write permission for other users from the default creation permissions.

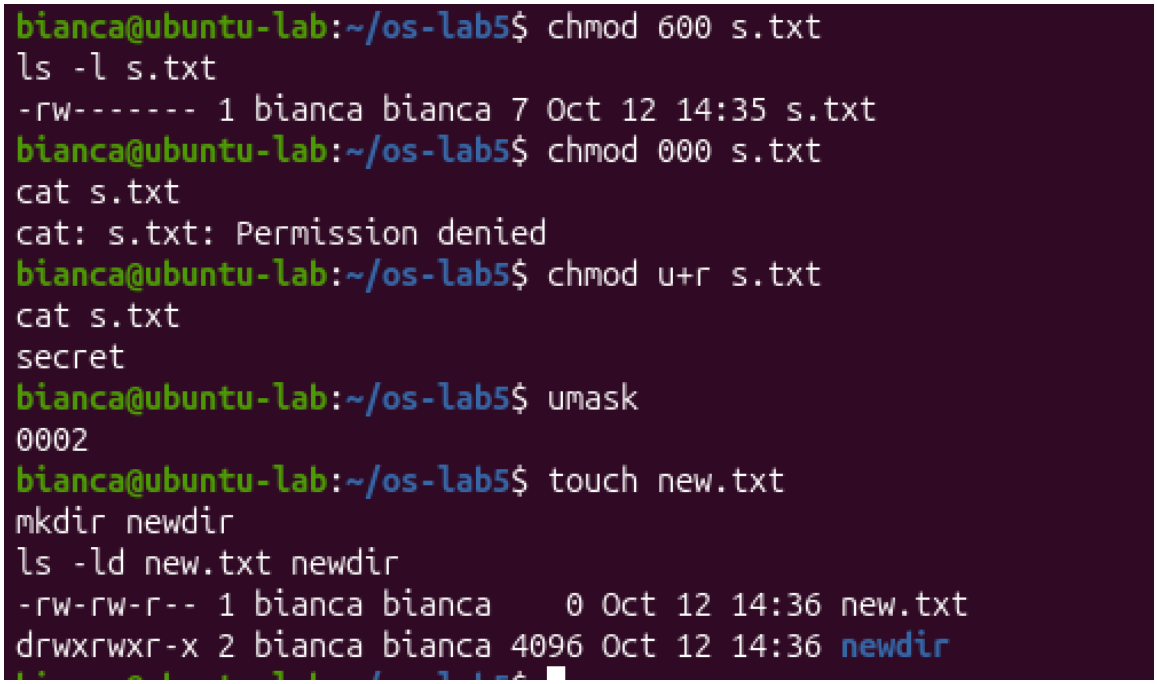


Figure 6: File permissions and default umask

5.4 Permission Modes

The main permission combinations are:

Mode	Permissions	Use
600	rw——-	Private file
644	rw-r–r–	Text document
755	rwxr-xr-x	Executable or directory
700	rwx——	Private directory

Each numeric digit represents a combination of read, write and execute permissions. The values are:

- read = 4;
- write = 2;
- execute = 1.

5.5 Directory Read and Execute Permissions

A directory and file were created:

```
mkdir box
echo hi > box/f
```

First, the directory was given read-only permission:

```
chmod u=r,go= box
ls box
cat box/f
```

The results were:

```
f
cat: box/f: Permission denied
```

The filename could be listed, but the file could not be accessed because the directory lacked execute permission.

Next, execute-only permission was assigned:

```
chmod u=x,go= box
ls box
cat box/f
```

The output was:

```
ls: cannot open directory 'box':
Permission denied

hi
```

The directory contents could not be listed, but the file could be accessed because its name was known.

Normal permissions were restored:

```
chmod 755 box
ls -ld box
```

The result was:

```
drwxr-xr-x ... box
```

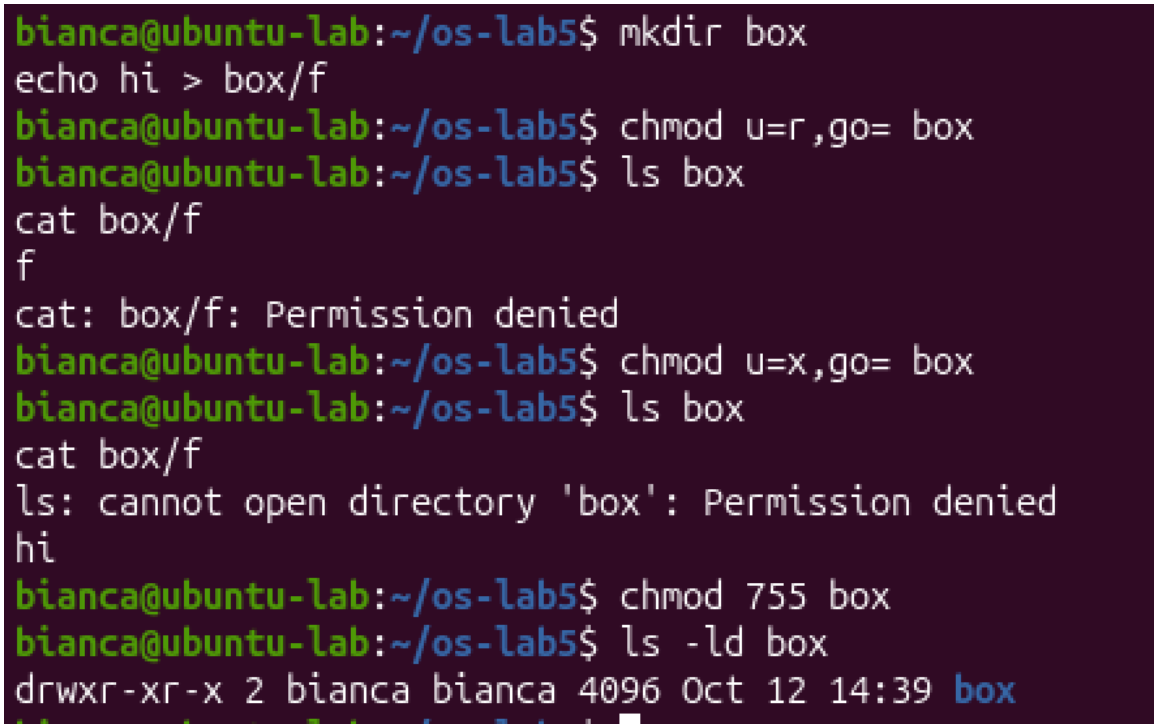


Figure 7: Read and execute permissions on directories

5.6 Special Permission Bits

The following command was executed:

```
ls -l /usr/bin/passwd
```

The output was:

```
-rwsr-xr-x 1 root root ... /usr/bin/passwd
```

The letter `s` indicates the setuid permission bit.
It allows the program to execute using the privileges of its owner.
Because `passwd` is owned by root, it can perform protected password updates.
The temporary directory was examined:

```
ls -ld /tmp
```

The output was:

```
drwxrwxrwt ... /tmp
```

The letter `t` indicates the sticky bit.
It restricts deletion and renaming of files in a shared directory.

5.7 Accessing a Protected System File

The following command was executed:

```
cat /etc/shadow
```

The result was:

```
cat: /etc/shadow: Permission denied
```

The permissions were inspected:

```
ls -l /etc/shadow
```

The output was:

```
-rw-r----- 1 root shadow ... /etc/shadow
```

The file was owned by root, and its group was shadow.
The normal user did not have permission to read this file.

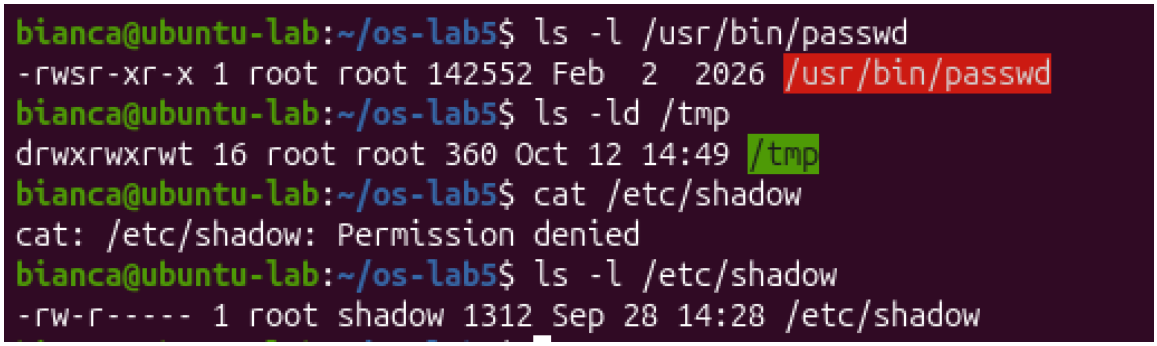


Figure 8: Setuid, sticky bit and protected system files

5.8 Part 3 – Observations

1. Permission modes 600, 644, 755 and 700 represent different combinations of read, write and execute access.
2. Ownership does not automatically provide read access. The owner must still have the corresponding permission.
3. The system umask was 0002. New files received mode 664 and directories received mode 775.
4. Read permission on a directory allows listing filenames. Execute permission allows accessing entries by known names.
5. The setuid bit on `/usr/bin/passwd` allows privileged password updates. The sticky bit on `/tmp` prevents unauthorized deletion or renaming of other users' files.

6 Part 4 – Disks and the Kernel as Files

6.1 Examining Disk Partitions

The command:

```
lsblk
```

was used to display block devices.
The important results were:

```
sda 25G disk
|-sda1 1G part /boot/efi
‘-sda2 23.9G part /
```

The virtual disk was named `sda` and had a total size of approximately 25 GB.
The disk contained two partitions:

- `sda1` – EFI system partition;
- `sda2` – root filesystem partition.

The disk structure was:

```
sda (25 GB)
|
|-- sda1 (1 GB)
| ‘-- /boot/efi
|
‘-- sda2 (23.9 GB)
    ‘-- /
```

6.2 Checking Disk Usage

The command:

```
df -h
```

displayed the disk usage information.
The root filesystem result was:

```
/dev/sda2 24G 7.7G 15G 35% /
```

This indicates:

- total size: 24 GB;
- used space: 7.7 GB;
- available space: 15 GB;
- disk usage: 35%.

The root filesystem had sufficient free space for additional files and applications.

```
bianca@ubuntu-lab:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  61.9M  1 loop /snap/core24/1644
loop2    7:2    0  19.9M  1 loop /snap/desktop-security-center/152
loop3    7:3    0  61.8M  1 loop /snap/core24/2125
loop4    7:4    0 247.5M  1 loop /snap/firefox/8762
loop5    7:5    0  90.7M  1 loop /snap/desktop-security-center/207
loop6    7:6    0 552.9M  1 loop /snap/gnome-46-2404/154
loop7    7:7    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop8    7:8    0 188.2M  1 loop /snap/mesa-2404/1836
loop9    7:9    0  79.9M  1 loop /snap/prompting-client/229
loop10   7:10   0  17.5M  1 loop /snap/prompting-client/221
loop11   7:11   0  43.4M  1 loop /snap/snapd/27709
loop12   7:12   0  10.9M  1 loop /snap/snap-store/1391
loop13   7:13   0   804K  1 loop /snap/snapd-desktop-integration/396
sda      8:0    0    25G  0 disk
├─sda1   8:1    0     1G  0 part /boot/efi
└─sda2   8:2    0  23.9G  0 part /
sr0     11:0    1  1024M  1 rom
bianca@ubuntu-lab:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           676M  2.3M  674M   1% /run
/dev/sda2        24G  7.7G   15G  35% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
efivarfs        256K   18K  239K   7% /sys/firmware/efi/efivars
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.7G  8.0K  1.7G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda1       1.1G  6.6M  1.1G   1% /boot/efi
tmpfs           338M   88K  338M   1% /run/user/1000
```

Figure 9: Disk partitions and filesystem capacity

6.3 Identifying the Filesystem Type

The following command was executed:

```
mount | grep " / "
```

The output was:

```
/dev/sda2 on / type ext4 (rw,relatime)
```

This confirmed that the root filesystem used the **ext4** filesystem.
The filesystem was mounted in read-write mode.

6.4 Examining CPU Information

The number of processors was checked using:

```
grep -c processor /proc/cpuinfo
```

The result was:

```
2
```

Ubuntu detected two logical processors.

Since the system was running inside VirtualBox, the number represented the processors assigned to the virtual machine rather than necessarily the physical CPU count of the host computer.

6.5 Examining Memory Information

Memory information was obtained with:

```
head -3 /proc/meminfo
```

The output was:

```
MemTotal: 3460128 kB
MemFree: 81664 kB
MemAvailable: 410384 kB
```

The total memory available to the virtual machine was approximately 3.3 GiB.

The available memory value indicated approximately 401 MiB of memory available for new applications at the time of the experiment.

6.6 Examining Process Information

The following command was used:

```
head -5 /proc/self/status
```

The output was:

```
Name: head
Umask: 0002
State: R (running)
Tgid: 9013
Ngid: 0
```

The process name was `head`.

This happened because `/proc/self` refers to the process accessing the file.

In this experiment, the `head` command opened the status file.

6.7 Examining File Descriptors

The following command was executed:

```
ls -l /proc/self/fd
```

The output showed file descriptors 0, 1, 2 and 3.

The first three descriptors represented:

- 0 – standard input;
- 1 – standard output;
- 2 – standard error.

They pointed to the terminal device:

```
/dev/pts/0
```

Descriptor 3 pointed to a directory inside `/proc`, opened by the `ls` process during execution.

This experiment demonstrated how Linux represents process resources using the virtual `/proc` filesystem.

```
bianca@ubuntu-lab:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
bianca@ubuntu-lab:~/os-lab5$ grep -c processor /proc/cpuinfo
2
bianca@ubuntu-lab:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        3460128 kB
MemFree:           81664 kB
MemAvailable:     410384 kB
bianca@ubuntu-lab:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   9013
Ngid:   0
bianca@ubuntu-lab:~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 bianca bianca 64 Oct 12 14:52 0 -> /dev/pts/0
lrwx------ 1 bianca bianca 64 Oct 12 14:52 1 -> /dev/pts/0
lrwx------ 1 bianca bianca 64 Oct 12 14:52 2 -> /dev/pts/0
lr-x------ 1 bianca bianca 64 Oct 12 14:52 3 -> /proc/9015/fd
```

Figure 10: Root filesystem, CPU, RAM and process information

6.8 Part 4 – Observations

1. The virtual disk was `/dev/sda`, with partitions `/dev/sda1` and `/dev/sda2`. The root filesystem was mounted on `/dev/sda2` using ext4.
2. The root filesystem was approximately 35% full, with 7.7 GB used and 15 GB available.
3. Ubuntu detected two logical processors and approximately 3.3 GiB of RAM.
4. The process name shown in `/proc/self/status` was `head` because that program accessed the virtual status file.

7 GitHub Repository

The laboratory files were managed using Git and uploaded to GitHub.

A local Git repository was initialized inside the `os-lab5` directory.

The following commands were used:

```
cd ~/os-lab5
git init
git branch -M main
git add report.md
git commit -m "Operating Systems Lab 5"
```

The remote repository was configured using:

```
git remote add origin https://github.com/
Biancatataroi/Operating-Systems-Lab5.git
```

The files were uploaded using:

```
git push -u origin main
```

Git confirmed that the local main branch was connected to the remote main branch.

The repository contains the Markdown laboratory report. The final PDF report and screenshots can also be added to the same repository.

The repository is available at:

https://github.com/Biancatataroi/Operating-Systems-Lab5

8 Conclusion

In this laboratory work, the Linux filesystem was explored through practical terminal experiments.

The first part focused on the Linux directory hierarchy and the different types of filesystem objects. Directories, regular files, symbolic links, character devices and block devices were identified using standard Linux commands.

The second part demonstrated how inodes and links work. It was observed that hard links share the same inode, allowing file data to remain accessible even after one filename is removed.

Symbolic links were found to behave differently because they store paths to target files and can become broken when their targets are deleted.

The experiments with directories showed how the number of hard links changes when subdirectories are created.

The difference between logical file size and physical disk allocation was also demonstrated using a small text file and a large sparse file.

The third part focused on Linux permissions and access control. Different permission modes were tested using `chmod`, while the `umask` command demonstrated how default permissions are assigned.

The directory permission experiments showed that read permission allows listing filenames, while execute permission allows access to files inside a directory.

Special permission bits such as setuid and the sticky bit were also examined through system files and directories.

Finally, the fourth part focused on storage devices and kernel information exposed through the Linux filesystem.

The commands `lsblk`, `df`, and `mount` were used to identify disk partitions, filesystem types and storage usage.

The virtual `/proc` filesystem provided information about logical processors, memory, running processes and open file descriptors.

Overall, this laboratory work provided a practical understanding of how Linux organizes files, manages storage, enforces permissions, and exposes operating system information through filesystem interfaces.